By TOUCH DEN · November 11, 2026

A First Look at C and Memory

Video 1 · Variables, Addresses, and Pointers

How Computers Work · Section 11

In this video, you will learn:

- Why security people learn a little C
- How a variable is a place in memory, with an address
- What a pointer is

Why a little C

C works directly with memory. Most languages (Python, Java) hide memory for safety; C lets you see and touch addresses and bytes.

That's why it matters here:

- The systems you attack/defend are often **C/C++** (OS kernels, core & IoT software).
- The most serious bugs, **buffer overflows** and relatives, live in C.
- It makes the stack and addresses **concrete**.

Goal: not to learn C, but to **see memory**.

A variable is a place in memory

```c
int price = 2500;
printf("price lives at %p\n", (void *)&price);
```

```
$ ./variables
price is 2500
price lives at address 0x16b376408
```

A variable has three things: a **name** (`price`), a **value** (2500), an **address** (where it lives). `&` gets the address.

> Memory attacks care about the **address and bytes**, not the name. (Addresses change each run, ASLR.)

Pointers, gently

A **pointer** is a variable that holds an **address**.

```
int *p = &price;   // p holds the address of price
printf("%d\n", *p); // *p = the value AT that address
*p = 3000;          // write 3000 there → changes price
```

```
$ ./pointers
value through pointer: 2500
price is now: 3000
```

We changed `price` **without naming it**, only through its address. Like a **slip of paper with a house address**: hold the slip, change what's inside.

Recap: variables and pointers

- C **works directly with memory**, that's why security uses it.
- A variable is a **name + value + address**; `&` gets the address.
- A **pointer** holds an address; `*p` reads/writes the memory it points at.
- Memory attacks are about **addresses and bytes**, not names.

Next video: buffers, overflows, and memory safety.

A First Look at C and Memory

Video 2 · Buffers, Overflows, and Memory Safety

How Computers Work · Section 11

In this video, you will learn:

- Buffers on the stack
- What a buffer overflow is (preview)
- Memory safety, and why languages differ

A buffer, used safely

A **buffer** is a fixed-size block of memory for a sequence of bytes.

```
char name[8];                  // 7 letters + end marker
strncpy(name, "Dara", sizeof(name) - 1);
name[sizeof(name) - 1] = '\0';
```

```
$ ./buffer
hello, Dara
```

Key words: **at most 7 bytes**. The buffer is fixed-size, but input can be **any** length. A careful program checks it **fits**.

What a buffer overflow is

Writing **more data than a buffer holds**, so it spills past the end over nearby memory.

```
char name[8];
strcpy(name, user_input);   // DANGER: no length check!
```

A 50-char input runs 42 bytes **past** the 8-byte buffer, over the **return address** on the stack (lesson 6).

> Overwrite the return address → the CPU jumps to the attacker's code. **Over-long data becomes control.**

The classic stack overflow (Binary Exploitation, in a lab).

Memory safety

Why do overflows exist? **C/C++ are not memory-safe:** they let a program write out of bounds or misuse a pointer, without stopping it.

Two responses:

- **Defenses on C/C++ code:** NX, ASLR, stack canaries (make exploitation harder).
- **Memory-safe languages** (Rust, Python, Java, Go): **prevent** the bugs entirely, they check or manage memory.

Why there's a strong push to write new security-sensitive code in **memory-safe** languages.

Recap: buffers and safety

- A **buffer** is fixed-size; not checking input length is the overflow bug.
- A **buffer overflow** can overwrite the **return address**, over-long data becomes control.
- **C/C++ aren't memory-safe**; languages like **Rust/Python** prevent these bugs.
- Defenses (NX, ASLR) make exploitation harder on existing code.

Next lesson: put it all together, explore a real running system by hand.